MINOR PROJECT REPORT

ON

"PHISHGUARD X: HYBRID PHISHING DETECTION SYSTEM"

Submitted To

National Forensic Sciences University

M.SC CYBER SECURITY

Submitted By

DIPANGSHU DEY

(250447002034)

Under the Supervision of

MR. HARSH PANCHAL

(LECTURER)

Submitted to

SCHOOL OF CYBER SECURITY & DIGITAL FORENSICS

NATIONAL FORENSIC SCIENCES UNIVERSITY

PONDA– 403401, GOA, INDIA.

[MAY,2026]

DECLARATION By STUDENT

I hereby declare that the work being presented in this Report titled ***"PhishGuard X: Hybrid Phishing Detection System"*** *by me i.e.* ***DIPANGSHU DEY****, having Enrolment Number* ***"250447002034"****, and submitted to the School of Cyber Security and Digital Forensic at National Forensic Sciences University, Gandhinagar; is my original work carried out during the period of* ***"January 2026"*** *to* ***"MAY 2026"*** *under the supervision of* ***"MR. Harsh Panchal".*** *I have complied it with the School's Ethical Code of Conduct and have duly acknowledged all sources through proper citations and references. The plagiarism check confirms that the overall similarity index is within 10%.*

_____

(Name & Signature of Student)

CERTIFICATE FROM GUIDE

*I hereby certify that the dissertation entitled "**PhishGuard X: Hybrid Phishing Detection System,** embodies the result of bonafide minor project work done by **Dipangshu Dey** of the Semester **II** having enrollment number **250447002034** for the degree of "**Master of science in Cyber Security**" in the "**School of Cyber Security & Digital Forensics**" of the "**National Forensic Sciences University, Ponda, Goa – India**" under my guidance and supervision. I further certify that this is an original work and that whatever material obtained and used from other sources has been duly acknowledged in the report. This work has not been submitted for any degree or diploma of any university or institute, as per my knowledge.*

Date:

Place: *Ponda, Goa*

Mr. Harsh Panchal,Lecturer
School of Cyber Security & Digital Forensics
National Forensic Sciences University,Ponda Goa–India, 403401.

CERTIFICATE

This is to certify that the minor project viva of ***Mr. Dipangshu Dey*** *of the* ***School of Cyber Security & Digital Forensics, National Forensic Sciences University, Ponda, Goa, India - 403401*** *was conducted on* ***11/05/2026*** *at the* ***National Forensic Sciences University, Goa Campus. His*** *Minor project titled "****PhishGuard X : Hybrid Phishing Detection System****" was evaluated and found satisfactory.*

Name & Signature of Internal Examiner 1 ***Name & Signature of Internal Examiner 2***

Date:

Place: *Ponda, Goa*

ACKNOWLEDGEMENT

I convey my deepest gratitude to my Project Guide MR. Harsh Panchal for his advice, knowledge, and guidance throughout the entire procedure of creation of this project. In addition to that, I am thankful to the Head of our Department, Dr. Panem Charanarur for providing me with all possible support, guidance, and encouragement. This has been of great help for me while executing the project successfully.

Lastly, I am thankful to everybody else whose contribution led to the accomplishment of this project

With Sincere Regards,

Dipangshu Dey

M.Sc. Cyber Security

ABSTRACT

PhishGuardX represents a multi-tiered hybrid phishing detection system designed as part of a cybersecurity minor project carried out at the National Forensic Sciences University, Goa. This system aims to detect any hidden phishing attempts within the given URLs, emails, SMS messages, files, and QR codes. It does so by employing a combination of six different methods such as rule-based URL analysis, machine learning classification, WHOIS domain age analysis, HTTP redirect chains analysis, content analysis of emails/SMS messages, and campaign detection into one hybrid scoring engine[1][2]

This entire application is coded using the Python language and comes with two separate interfaces - a Tkinter-based Graphical User Interface for instant scanning operations and a SOC dashboard developed using Streamlit for continuous monitoring along with historic data analysis. Both these interfaces share the same backend engine.[6]

The machine learning module makes use of a Random Forest classifier made up of 200 decision trees based on a set of phishing and legitimate URLs that have been labeled in advance. Eight features obtained from the URL are sent to the classifier for processing per each input. Additionally, the campaign detection feature tracks all scanned domains and marks the occurrence of any coordinated phishing attempts if similar domains are detected across three or more different URLs. In case of QR code input, OpenCV scans the URL and then analyzes it using the complete set of rules defined above.[3][5]

Each analysis will generate an automated professional PDF report created with the help of ReportLab library that includes an executive summary, WHOIS intelligence, ML analysis, rule-based output, redirect chain trace, content analysis, QR code analysis if applicable, MITRE ATT&CK technique mapping, and a SHA-256 hash value of evidence.[7][9]

Keyword: Phishing Detection, Machine Learning, Random Forest, Url Analysis, WHOIS, Redirect Chain, Campaign Detection, Qr Code Analysis, Analysis Report, Streamlit, Tkinter, Sqllite ,MITRE ATT&CK, Python

.

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
CSV	Comma Separated Values
DB	Database
DFD	Data Flow Diagram
GUI	Graphical User Interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
JSON	Javascript Object Notation
ML	Machine Learning
MITRE ATTT&CK	Adversarial Tactics, Techniques, And Common Knowledge Framework
PDF	Portable Document Format
QR	Quick Response (Code)
RF	Random Forest
SHA	Secure Hash Algorithm
SMS	Short Message Service
SOC	Security Operation Center
SQL	Structured Query Language
TLD	Top Level Domain
UI	User Interface
URL	Uniform Resource Locator
WHOIS	Who Is (Domain Registration Protocol)

LIST OF TABELS

LIST OF FIGURES

Table of Contents

CHAPTER 1 INTRODUCTION

1.1 Introduction and Background

The internet has become a fundamental aspect of our modern working, communicating, and financial management styles. In view of this ever-growing reliance on digital means, phishing has emerged as one of the most common and damaging types of cyberattacks. Essentially, phishing involves tricking the user into clicking on malicious links, entering their credentials on fake websites, or downloading harmful programs, while still maintaining the belief that they have interacted with an authentic source. According to the cybersecurity industry statistics, billions of phishing emails and SMSs are sent yearly, with losses reaching hundreds of billions of dollars worldwide.[4] The problem with phishing goes far beyond its technological nature – it manipulates human psychology using urgency, threats, and deception. The hackers usually disguise themselves as banks, authorities, popular applications, and reliable brands.

Current phishing detection techniques mainly employ three distinct strategies: blacklist checking (ineffective against new domains), independent machine learning models (which can be attacked using malicious inputs), and manual inspection (inadequate at a large scale). Alone, none of the three methods is effective enough to solve the problem in question.[11][12]

The emergence of phishing via QR codes (quishing), which became quite popular in recent years, creates additional security issues, since in this way a victim gets redirected to the phishing page without having any prior contact with the URL.[14]

For this reason, PhishGuardX has been designed to address the problem effectively. It is not simply a URL checking tool – it is a fully-fledged hybrid phishing detector that combines several detection algorithms, provides professional analyses, and includes a desktop application and SOC web interface.

1.2 Problem Statement

The design of PhishGuardX emerged from several main issues:

• Currently existing methods conduct assessments based solely on analyzing individual links and do not consider the repeated use of similar domains in other phishing URLs; therefore, it is impossible to identify coordinated attacks.

• QR code phishing (or quishing) has recently become increasingly popular, and at the same time, many free detection utilities cannot scan QR codes.

• Blacklisting mechanisms are unable to prevent phishing attempts originating from freshly created websites, as their information is not included in any database.[15]

• The content of emails or SMS messages is not assessed along with the link; thus, elements like urgency indicators, impersonation, and call-to-action language may be missed.

• Security professionals do not have an all-inclusive program that provides for detection, analysis, and evaluation of the collected data.

• There is currently no open-source solution for creating an efficient scoring system that would incorporate machine learning predictions, rules, domain age assessment, and redirect chains.

1.3 Aim and Objectives

The aim is to design and develop a hybrid and layered phishing detection tool called PhishGuardX. In this respect, PhishGuardX would analyze the URLs, emails, SMS, files, and QR codes using several layers such as machine learning techniques, rule-based, WHOIS data, redirect chain tracer, and content analysis. As for the analysis, a professional report would be created for each.

The objectives can be described below:

- Creation of a rule-based URL analysis module for identifying potential risks like suspicious keywords, IP addresses, imitation patterns, questionable TLDs, URL shorteners, lack of HTTPS protocol usage, and URL hex encoding.

- Use of a Random Forest model with 200 estimators for prediction of phishing risks for a certain URL based on eight derived features.[3][5]

- Implementation of the WHOIS domain age checker to identify those domains that are under thirty days old or registered recently (within the period of less than one hundred eighty days)[20]

- Construction of the redirect-chain tracer using Python requests library to trace all hops during HTTP process and classify each of them as either OK, WARNING, SUSPICIOUS, ALERT, and UNREACHABLE.

- Creating a content analyzer which would analyze the body of emails/SMS/files for presence of suspicious keywords and terms connected with urgent action, imitation-related terms, and calls to do dangerous things.

- Development of the campaign detector implemented through SQLite for tracking the history of scanned domains and triggering an alert when a specific URL has appeared three times or more in different URLs.[10]

- Providing the functionality for analyzing a QR code using OpenCV to decode its image content and analyzing this obtained URL.[8]

- Creation of a hybrid risk assessment algorithm for combining all results of other modules into one score ranging from zero to one hundred.

- Generating professional reports in PDF format using ReportLab containing thirteen sections and including the MITRE ATT&CK framework and evidence hash (SHA-256).[7][9]

- Creating two different user interfaces: Tkinter based desktop application and Streamlit-based SOC dashboard application**.** [6]

1.4 Scope of the Project

PhishGuardX is intended for use by security analysts, students, and SOC analysts to determine if there are any phishing threats in a particular URL, email, SMS, file, or QR code. The features of interest in this project include:

- Input Types: URLs, emails, SMS, file paths (any text files), and QR codes.
- Detection Engines: Six detection engines are incorporated into one hybrid scoring engine.
- Machine Learning: Pre-trained random forest machine learning model (url_model.pkl) is loaded at startup and utilized in URL prediction.
- Database Engine: Scanned URLs are logged in a SQLite database (campaigns.db) containing domain name, verdict, and date/time stamp of the scan.[10]
- Two GUI Applications: There are two graphical user interface applications – Tkinter (gui.py) and Streamlit (dashboard.py) communicating with the same engine.[6]
- Reporting: For every scan performed, a PDF report is generated and stored in the reports/ folder.[7]
- Local Application Only: It does not connect with any external services like VirusTotal or Shodan.
- Passive Scanning Only: It does not scan or execute any active content on the website.

CHAPTER 2 LITERATURE SURVEY

2.1 Study Of Existing System

Phishing detection methods have been actively researched and developed for over two decades now The most common method of detecting phishing sites involves the usage of blacklists which are maintained by companies like Google Safe Browsing and all modern-day web browsers. Such detection works quickly and efficiently in regards to phishing URLs that have already been identified and registered in the database; however, this system doesn't work when a new phishing website has been created, since those kinds of URLs aren't in the database yet. Attackers always try registering a new domain.[15]

A lot of work has been done on machine learning-based detection of phishing in academic literature. These algorithms usually rely on the extraction of numerical characteristics of the URL such as its length, number of subdomains, inclusion of any IPs, and whether a suspicious TLD is involved. Machine learning can be highly efficient in isolation; however, it usually lacks the explainability factor (it's hard to tell why an algorithm determined a URL to be suspicious) and other signals such as emails or redirects are not taken into account.[1][2][11] Products for enterprise-level email security, such as Proofpoint or Mimecast, can do content scanning but require purchasing commercial licenses and closed source code which can't be used by independent researchers.[12]

There also isn't a tool providing a detailed report about phishing detection for research purposes. In regards to quishing attacks, there practically is no accessible tool for phishing detection using QR codes. Users scan these codes through their phone cameras while being completely unaware of what kind of URL will be opened when clicking on it.[14]

2.2 Problems in Existing Systems

• Blacklists do not detect recently registered phishing URLs that have not yet been flagged.[15]

• Machine learning alone does not take into account the age of the domain, redirect, and content signals.[1][11]

• None of the currently available open source phish detectors support quishing.

• Many existing phishing tools lack historical data about the domain and fail to identify campaigns

• There are no available phishing reports from any existing tools which can be used for audits or incident response purposes.

• Most open-source phish detectors lack MITRE ATT&CK mapping.[9]

• URL shorteners are commonly used to obscure the malicious destination site, yet few tools assign penalties to such URLs.

• Email content signals (phrases of urgency, impersonation, CTA) are scarcely considered alongside the URL.

2.3 Requirements of New System

In line with the discussion of the current state of affairs regarding available tools and their drawbacks, the following requirements have been identified for PhishGuardX[12][13]

• The tool should accept various input sources, including URLs, emails, SMSes, files, and even QR code scans.

• The application cannot use blacklists exclusively and should generate its own risk assessment based on several detection signals.

• Machine learning cannot serve as the only detection method; rather, it should become one of the layers used in detecting phishes.[1]

• Detection of domain age using WHOIS should occur in order to identify newly registered phishing domains.[20]

• The complete HTTP redirect chain should be detected since the latter could be used to evade phishing detection systems.

• Analysis of email/SMS content should take place in order to search for phishing language and other signs of urgency.

• Decoding of the QR code image, followed by the detection of a URL hidden in the image, should be performed.[8]

• All detected URLs should be logged for pattern identification.

• PDF-based analysis report must be issued with each analysis run.[7]

• Two interfaces – a desktop GUI and a web version – should be developed.[6]

2.4 Feasibility Study

2.4.1 Technical Feasibility

PhishGuardX is achievable from a technical standpoint since all needed technologies and libraries are freely available in Python. Python 3.8+ is compatible with all leading operating systems. Scikit-learn is responsible for Random Forest classification.[5] The requests library is used for HTTP redirection analysis. The python-whois library contains the details of the domain name registration. OpenCV offers QR code detection without any external dependencies. SQLite3 is integrated into the standard library of Python. ReportLab is an established PDF generation tool. Streamlit offers a quick web interface that can be launched on a local computer using one command. Tkinter is available in Python's standard library.

2.4.2 Operational Feasibility

The tool has operational capabilities such that it can be used by cybersecurity students, analysts, and a small SOC team. Upon installing the Python modules necessary for the system, the system will operate using one command either python gui.py or streamlit run dashboard.py without any server configurations or cloud deployment needed. The database used to store the data is SQLite while all the reports generated by the tool will be stored in a local folder.

2.4.3 Economic Feasibility

The complete framework is created using entirely free and open source technologies. There are no licensing costs, no charges for the use of any APIs, and no subscriptions needed. All that is needed is a computer capable of executing Python 3.8+. This makes the tool economically feasible for educational purposes as well as for small organizations or individuals.

~

2.5 Tools and Technologies Required

Table 1: Tools and Technologies Used in PhishGuardX

Tool / Technology	Purpose in PhishGuardX
Python 3.8+	Primary programming language for all modules
scikit-learn	Random Forest classifier for ML-based URL prediction
joblib	Loading and saving the trained ML model (url_model.pkl)
requests	HTTP redirect chain tracing
python-whois	WHOIS domain age and registration date lookup
OpenCV (cv2)	QR code image decoding for quishing detection
SQLite3	Local database for URL history and campaign detection
ReportLab	PDF analysis report generation
pypdf	Merging cover page and body PDF into final report
Streamlit	Web-based SOC dashboard interface
Plotly / Plotly Express	Interactive charts on the Analytics dashboard
pandas	Dataframe handling for scan history queries
Tkinter	Desktop GUI interface (gui.py)
threading	Running backend analysis in a non-blocking thread in the GUI
hashlib	SHA-256 hash generation for report integrity
re (regex)	URL pattern matching, IP detection, encoding detection
urllib.parse	Extracting hostname and domain from URLs

CHAPTER 3 SYSTEM DESIGN

3.1 System Architecture

PhishGuardX employs a modular multi-layered design approach in which different detection concerns are performed separately within unique Python modules. All results are aggregated through a centralized hybrid scoring system inside analyzer.py. The architectural layers include five.

Layer 1 - Input Layer: This layer includes the Tkinter GUI (gui.py) and Streamlit dashboard (dashboard.py). Here, users define the type of input and the target to be analyzed. When using QR codes, the image will first be processed in qr_scanner.py and the detected URL fed to analyze_url().

Layer 2 - Pre-processing Layer: The _sanitize_input() function inside analyzer.py takes care of input sanitization, such as stripping whitespaces, testing against blocked URL schemes, and issuing a warning in case there is no http/https scheme.

Layer 3 - Detection Layer: The detection process comprises six independent steps including: rule-based analysis, redirect chain analysis, content analysis, ML prediction, WHOIS domain age

analysis, and campaigns identification.

Layer 4 - Scoring Layer: The hybrid scoring is calculated through the equation: Final Score = (rule_score × 0.4) + (redirect_score × 0.4) + (content_score × 0.4) + (whois_score × 0.4) + (ml_score × 0.6) [Maximum Value 100]. Verdicts: SAFE (0-29), SUSPICIOUS (30-69), PHISHING (70-100).

Layer 5 - Output Layer: The final result is provided as a Python dictionary. The output is displayed in the UI of both interfaces. The ReportGenerator in reporter.py will create a PDF report based on the same dictionary

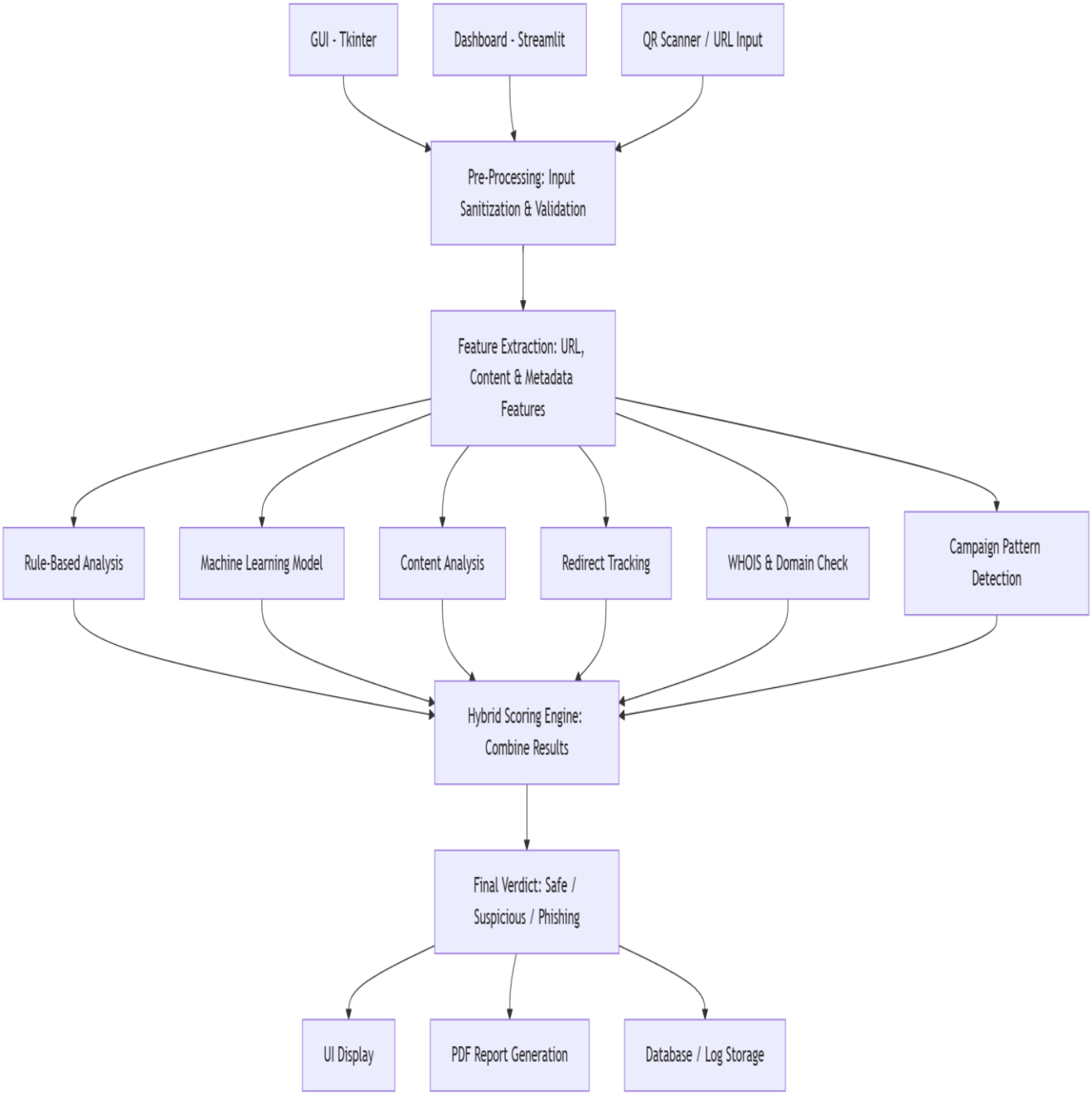

Figure 1:System Architecture of PhishGuardX

3.2 Use Case Diagram

As depicted by the use case diagram presented below, PhishGuardX deals with five types of input as well as three types of output which include the verdict displayed on screen, the PDF report and the information stored in the database respectively.

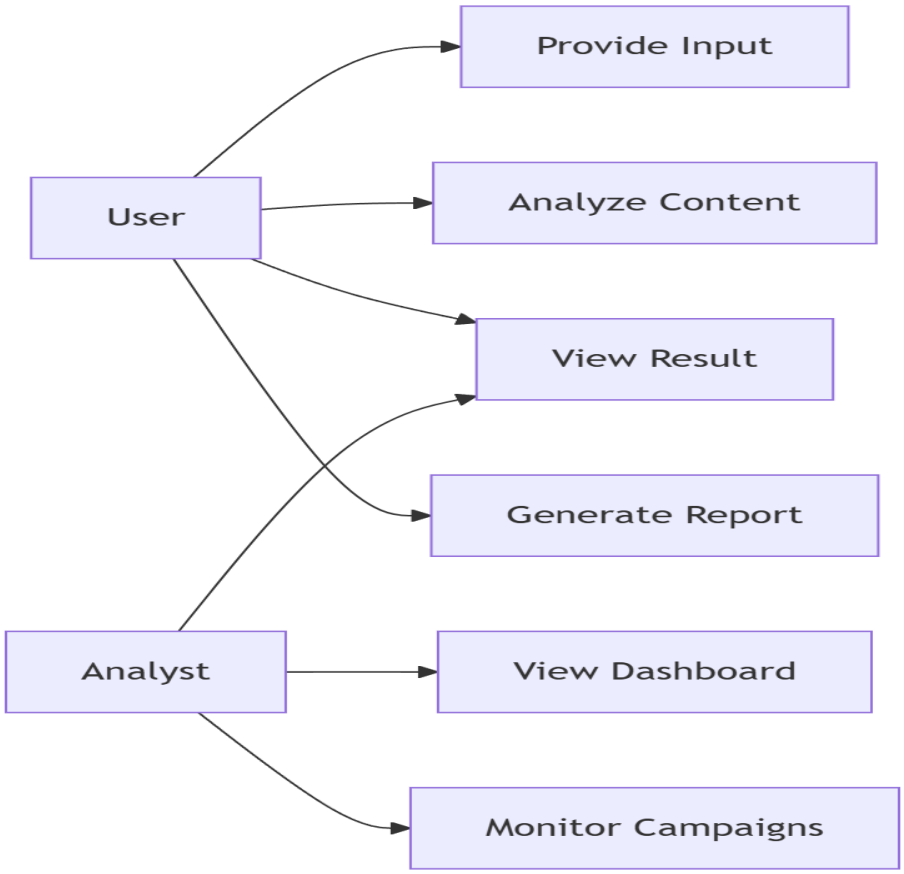

Figure 2: Use Case Diagram

The figure above indicates that the analyst is able to scan any URLs, emails, SMS texts, files, and QR codes, basically all input where each scan invokes the complete detection process. The dashboard provided by Streamlit will also enable analytics views and campaign tracking.

3.3 Module Description with Class Diagram

3.3.1 analyzer.py - The Core Detection Engine

Here lies the core of the entire analysis process. This is where the ThreatAnalyzer class is implemented, which controls the entire process. The model is loaded when the program starts up. analyze_hybrid() method executes the whole process of detection through six steps while _rule_analysis() checks URLs, _content_analysis() scans emails or SMSes, and _ml_analysis() performs prediction using Random Forest.

3.3.2 feature_extractor.py - ML Feature Engineering

The current module picks exactly 8 numbers out of any URL as its inputs to the ML algorithm. The eight numbers are: Length of URL, Number of dots, Number of hyphens, HTTPS flag (binary), IP flag (binary), At flag (binary), Length of domain name, and social engineering keyword count [5]

3.3.3 redirect_tracker.py - Redirect Chain Analysis

The present module leverages Python's requests library to trace a given URL through its entire redirection path,with a maximum of 10 hops, where each hop is labeled as follows: OK, SUSPICIOUS (famous URL shortener), (not encrypted), ALERT (redirection cycle), UNREACHABLE, TIMEOUT, or ERROR

3.3.4 campaign_detector.py - Campaign Intelligence

This module deals with the SQLite database called campaigns.db, which contains information about all the URLs scanned. The detect_campaign() method checks the number of occurrences of the same domain. Once the count becomes equal to three or higher, a campaign warning is issued. The store_url() method avoids any duplicate entry for the same URL.[10]

3.3.5 qr_scanner.py - QR Code Analysis

Module that detects quishing.decode_qr() function leverages QRCodeDetector from the OpenCV library in three iterations: default, grayscale, and Otsu's thresholding. Regular expression finds valid http(s):// URLs in the QR content. When the scanned QR platform is whitelisted, it returns a SAFE verdict immediately; otherwise, it goes to analyze_url().[8]

3.3.6 reporter.py - PDF Report Generator

In this module, a comprehensive report in the form of a multi-page PDF document is created using the ReportLab framework that consists of 13 sections including Executive Summary, Target Info, WHOIS intelligence, ML Analysis, Rules-Based Analysis, Redirect chain, Content Analysis, QR Code Analysis,

Campaign Intelligence, Attack Simulations, Recommendations, Execution Log, and Cryptographic evidence integrity (SHA-256).[7][9]

3.3.7 gui.py - Tkinter Desktop Interface

The module features a native desktop GUI. Analysis occurs on a separate thread via Python's threading library to ensure that the UI stays responsive. Features of the GUI include color-coded verdicts, risk scores, ML confidence, attack type, execution pipeline, and analysis log data.

3.3.8 dashboard.py - Streamlit SOC Dashboard

Module for web dashboard for professionals consists of three pages that include Threat scanner (main page with animated pipeline and results), Analytics (page with historical graphs generated by Plotly), and Campaign monitor (page showing all currently active campaigns).[6]

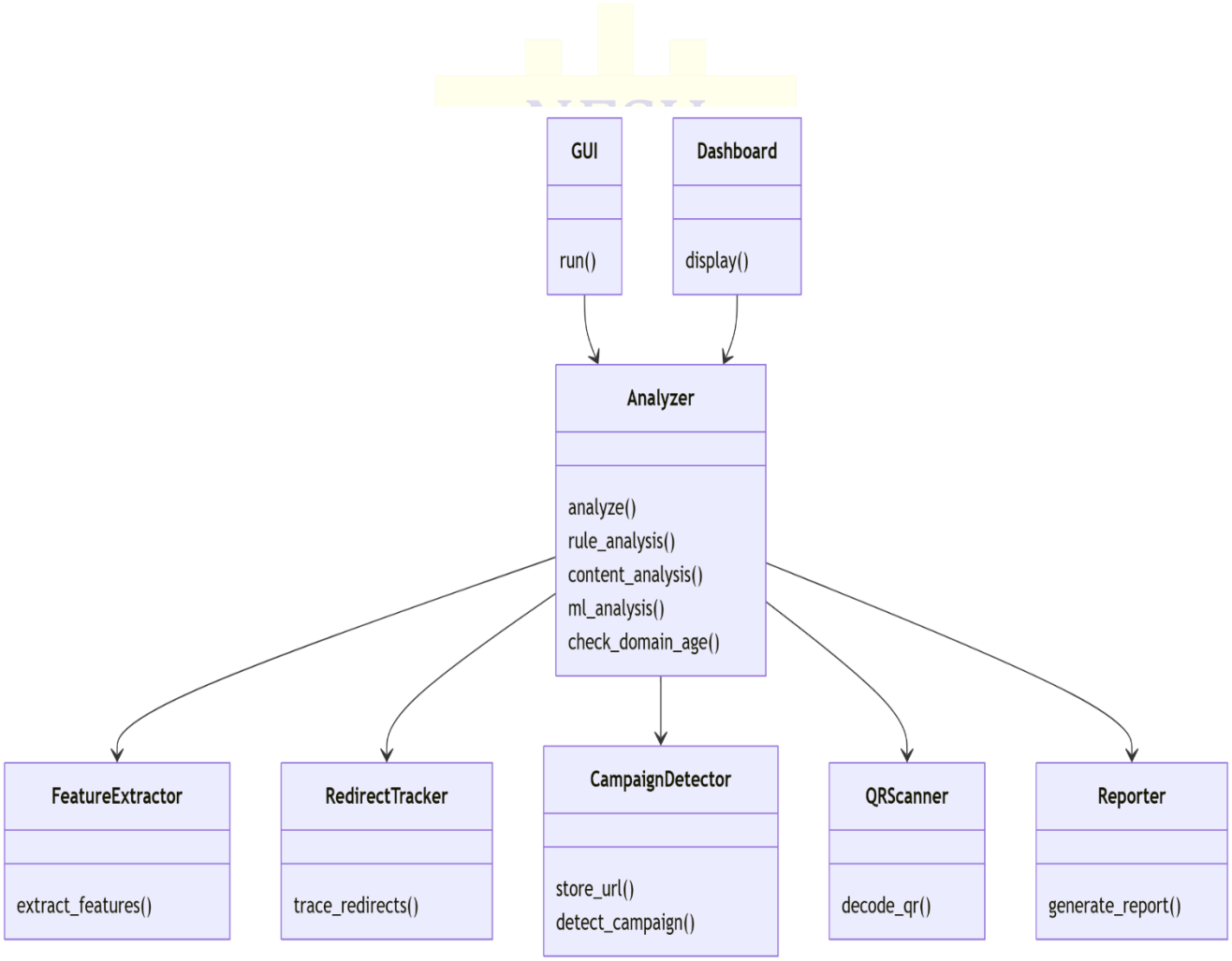

Figure 3: Class Diagram

3.4 Data Flow Diagram

3.4.1 Level 0 - Context Diagram

At the highest level, PhishGuardX will take inputs from the User (URL, Email, SMS, File or QR Code) and provide the following three outputs: Verdict and Risk score on-screen, PDF Analysis Report on Disk, and an Entry into the database file campaigns.db

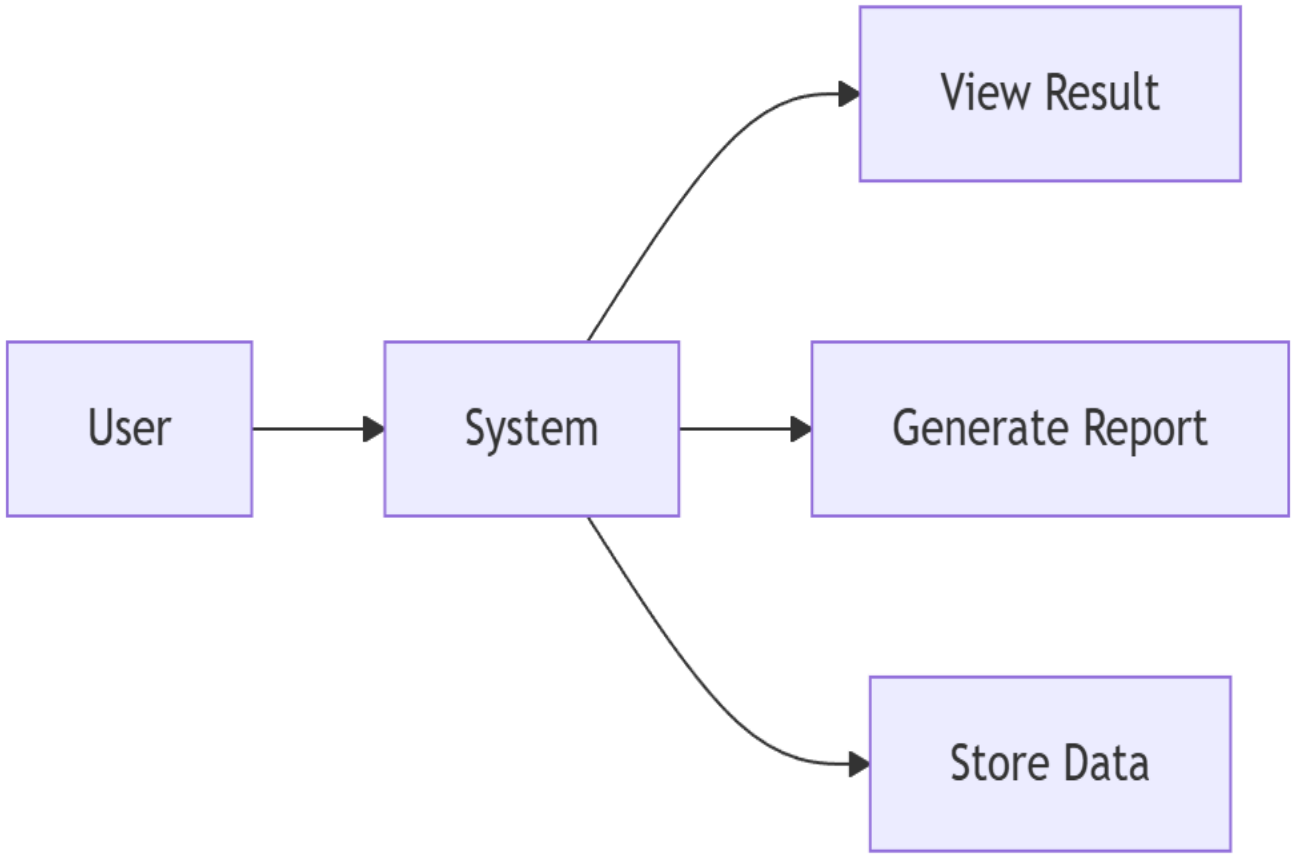

Figure 4: Data Flow Diagram — Level 0 (Context Diagram)

In the Level 0 diagram, PhishGuardX is represented by a process box that accepts inputs from an external entity called 'User' and generates three outputs in the form of Screen Output, PDF Report, and Database Record

3.4.2 Level 1 - Detailed Data Flow

The detailed data flow is as follows: User Input > Input Sanitization > Database Storage (verdict = UNKNOWN) > Campaign Identification > Trusted Domain Validation > Rule Evaluation > Redirect Trace > Content Examination > Machine Learning Prediction > WHOIS Lookup > Hybrid Scoring > Decision Making > Database Update > Report Creation > UI Display

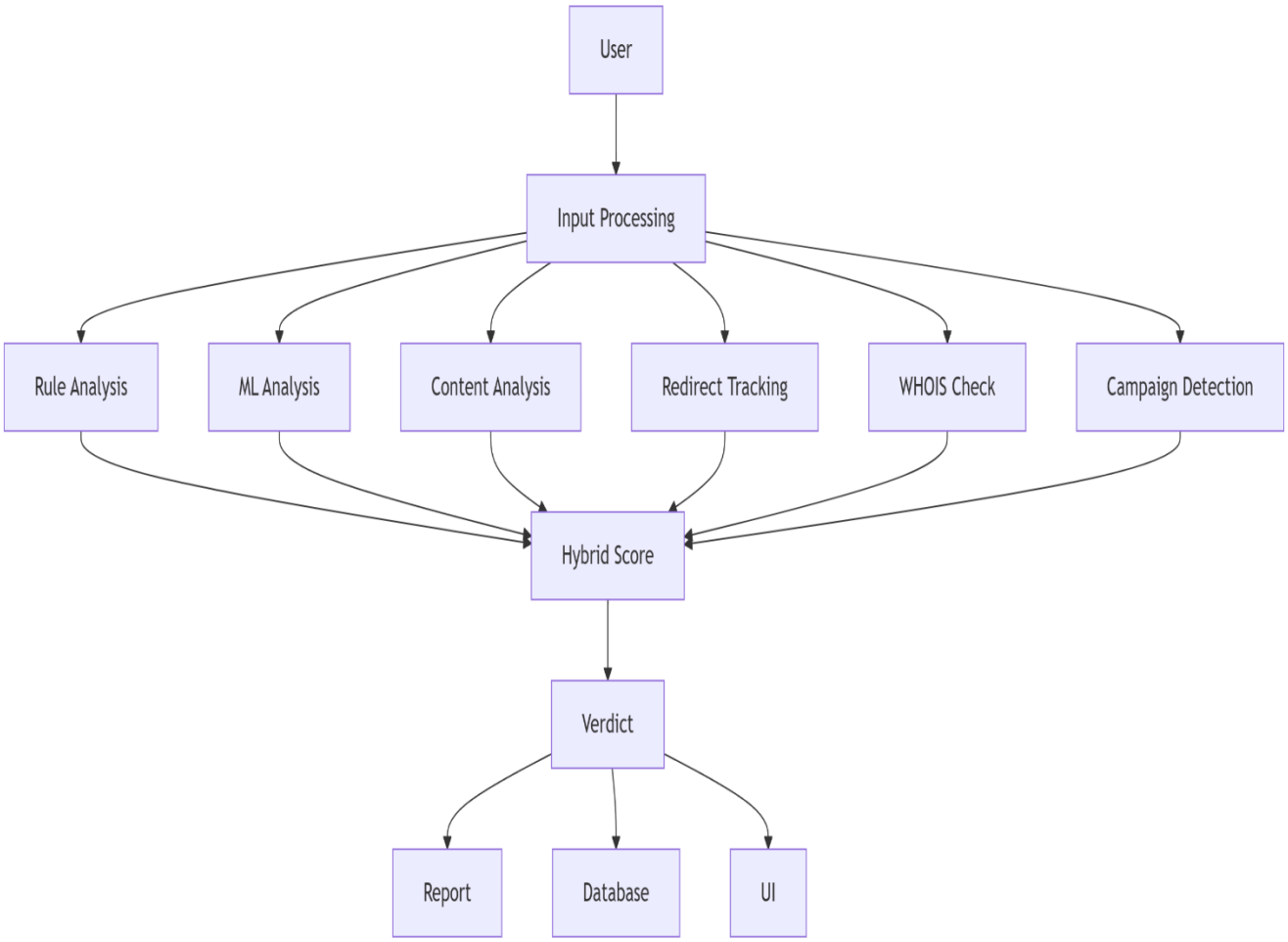

Figure 5: Data Flow Diagram — Level 1 (Detailed)

Level 1 DFD indicates the movement of data through all the six detection modules sequentially comes together at the hybrid scoring stage, and then gets divided between the UI rendering branch and the report generation branch.

3.5 System Flowchart

The diagram below depicts the full process of decision-making for a particular analysis request from the validation of input all the way through each level of detection up to the determination of the verdict and generation of the report

The flow chart illustrates the decision points like the following: Is this a trusted domain? (quick access to SAFE), Is this a QR input? (process through qr_scanner first), Is the redirection tracking successful? (handle connection exceptions smoothly). This flow concludes with the verdict assignment and PDF report creation

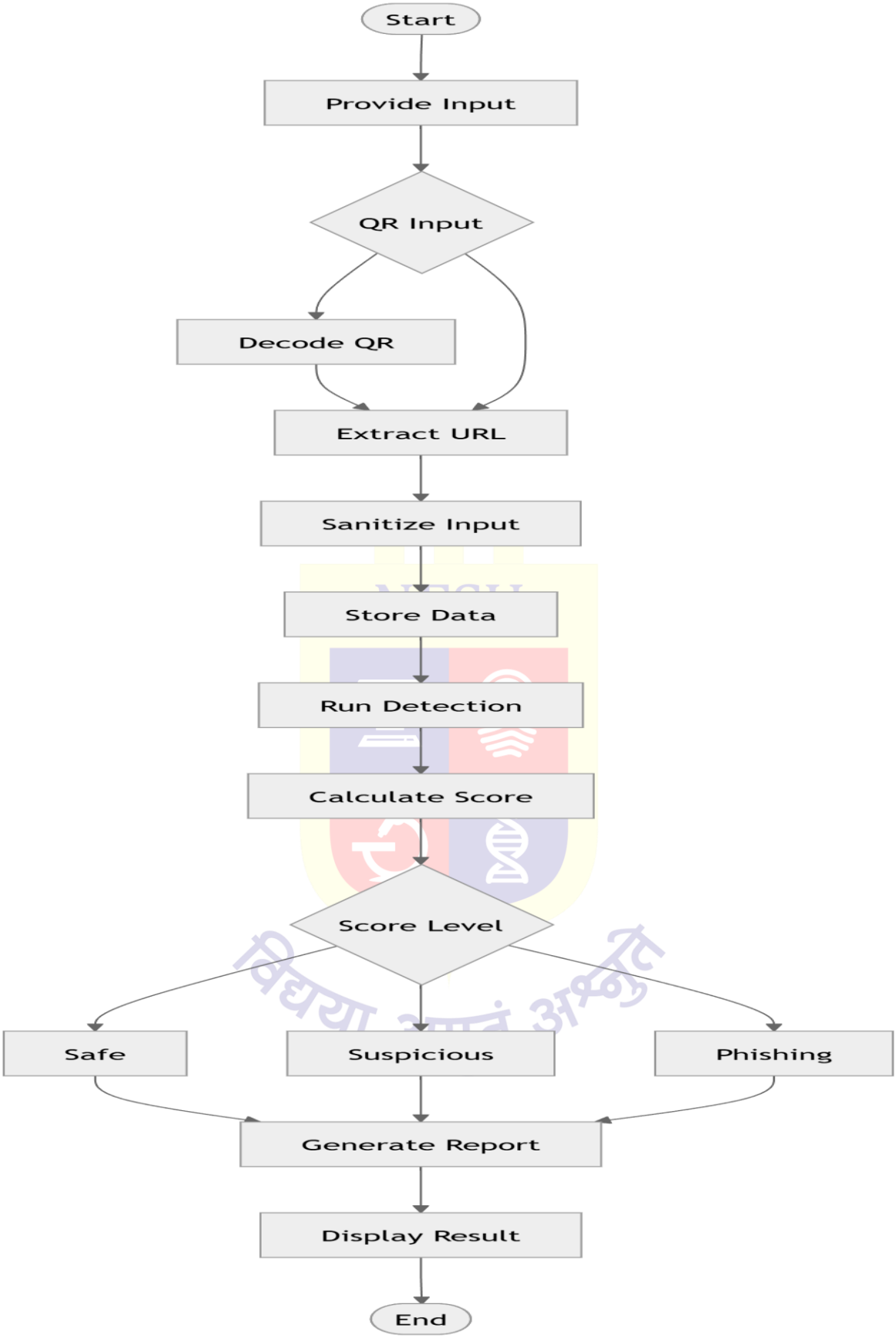

Figure 6: System Flowchart

3.6 Sequence Diagram

The sequence diagram illustrates the order of interaction among the key components during one scan procedure, starting from when the user initiates the process until when the verdict is shown on the screen.

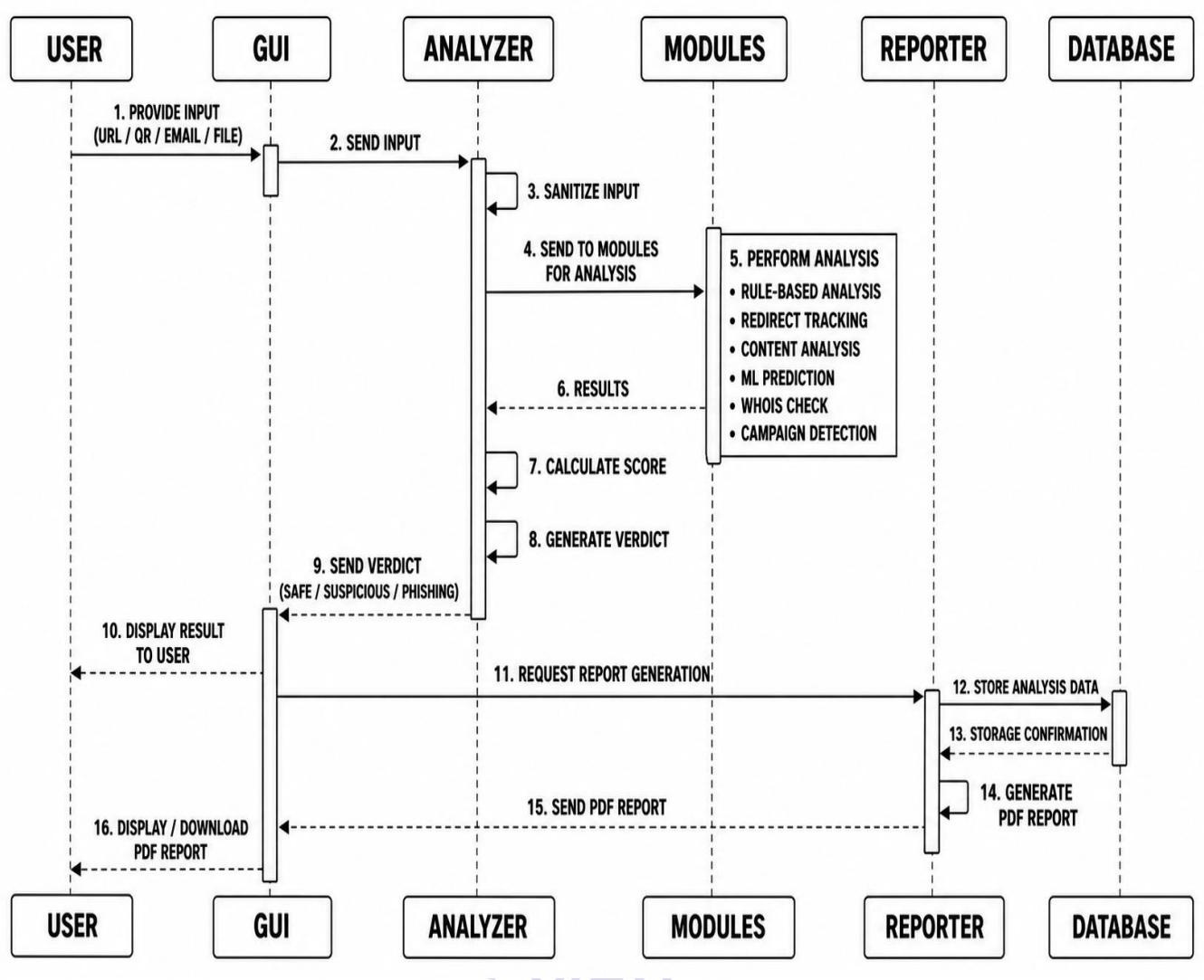

Figure 7: Sequence Diagram

The sequence diagram indicates that the GUI or Dashboard invokes analyze_url(), which in turn invokes feature_extractor.py, redirect_tracker.py, campaign_detector.py, and check_domain_age() in analyzer.py. The data is processed, and scores are generated and sent back to the GUI, whereupon it invokes reporter.py to create the PDF file.

3.7 Database Design

PhishGuardX uses a single SQLite database file named campaigns.db. The database consists of one table:[10]

Table 2: Database Schema - campaigns.db

Column	Data Type	Description
id	INTEGER (PK)	Auto-incrementing unique identifier
domain	TEXT NOT NULL	Domain extracted from the URL using urlparse
url	TEXT NOT NULL	The full URL or placeholder (email-input, sms-input)
verdict	TEXT DEFAULT 'UNKNOWN'	Set to UNKNOWN initially, updated after analysis
timestamp	TEXT NOT NULL	Datetime in format YYYY-MM-DD HH:MM:SS

The URL is added prior to any processing (verdict set to UNKNOWN) and updated after the final verdict has been reached. This dual-step process guarantees that the current URL is included in the count during the threshold test for campaign detection. Duplicate URLs are avoided using SELECT COUNT prior to adding the URL

CHAPTER 4 IMPLEMENTATION

4.1 Implementation Environment

Table 3: Implementation Environment Setup

Component	Configuration / Setup
Operating System	Windows 10/11 or Ubuntu 22.04+
Programming Language	Python 3.8+
Desktop GUI Framework	Tkinter (built into Python standard library)
Web Dashboard Framework	Streamlit 1.28+
ML Library	scikit-learn 1.2+ with joblib
Database	SQLite3 (built into Python standard library)
PDF Generation	ReportLab 4.0+ with pypdf
QR Code Decoding	OpenCV 4.5+ (cv2)
HTTP Requests	requests library 2.28+
WHOIS Lookup	python-whois library
Charts & Visualisation	Plotly 5.0+ and Plotly Express
Data Handling	pandas 1.5+
IDE Used	VS Code / PyCharm

4.2 Coding Standards

The following coding standards have been observed during the development of PhishGuardX:

- All Python code conforms to PEP 8 style conventions, with 4 spaces being used for indentations.
- Every module must have a single, well-defined responsibility, where, for example, the detector is responsible for analysis, campaign_detector for database interaction, and reporter for generating reports.
- Docstrings describing the role of each public function have been included to clarify its parameters and return value(s).

- Lazily initializing all module level singletons (_analyzer_instance, etc.) so as to not incur startupcosts.Exceptions are used wherever appropriate to ensure that a single failure, such as a timeout while doing WHOIS queries, does not bring down the entire analysis.
- Connections to databases are created and destroyed inside the same function call to avoid leaking them.
- For thread safety of the Tkinter-based GUI implementation, self.root.after(0, callback) pattern has been used for all callbacks from background threads.
- Using a scan_id technique in Streamlit for session management to avoid showing stale results after starting a new scan.0

4.3 Module-wise Implementation

4.3.1 Rule-Based Detection Engine

The function _rule_analysis() in analyzer.py evaluates the URL string using 10 separate analyses, resulting in a maximum possible rule score of 100.[1][2]

Table 4: Rule-Based Detection Signals and Their Weights

Detection Rule	How It Works	Score Added
Suspicious Keywords	Checks for: urgent, verify, secure, bank, account, login, update, password	+10
Missing HTTPS	URL does not start with https://	+10
IP Address Used	Regex detects any IPv4 address pattern in URL	+20
Too Many Subdomains	Hostname contains more than 3 dots	+10
Long URL	Total URL length exceeds 75 characters	+10
Hex Encoded URL	Detects %XX encoded characters in URL	+15
URL Shortener	Hostname contains bit.ly, tinyurl, or t.co	+15
Brand Impersonation	URL contains a brand name but hostname does not match real domain	+30
Suspicious TLD	.xyz .top .tk .ml .ga .cf .click .link .online .site etc.	+20
Fake Domain Pattern	3+ hyphens, brand in subdomain, or domain >50 chars	+20

4.3.2 Content Analysis Engine

The function _content_analysis() inspects emails, SMS messages, or file bodies for signs of phishing in four categories:

Table 5: Content Analysis Keywords and Phrases

Category	Examples	Max Score
Phishing Keywords	urgent, verify, login, password, account, bank, suspended, confirm	+12
Urgency Pressure Phrases	immediately, within 24 hours, action required, expires soon, final notice	+10
Impersonation Terms	paypal, apple, microsoft, google, dear customer, support team, security team	+10
Suspicious CTA Phrases	click here, verify now, login now, click the link, reset your password	+8

The prediction of probability of each class can be obtained using predict_proba() function of the model. Confidences are then calculated by multiplying the maximum value of the probabilities by 100. In case the ML model detects a phishing website, then ml_score of 60 is used to calculate the score for the hybrid system. The weight for the ML component is 0.6.

4.3.3 Machine Learning Model

ML model used is a random forest classifier with 200 decision trees trained using scikit-learn with a labelled data set consisting of phishing and non-phishing websites. The model is serialized as url_model.pkl using joblib and loaded during startup time. The following are the features extracted for each URL in function extract_features() of feature_extractor.p. [3][5]

Table 6: ML Feature Set Extracted per URL

#	Feature Name	What It Detects
1	URL Length	Very long URLs are common in phishing
2	Subdomain Count (dot count)	Phishers use many subdomains to mimic brands
3	Hyphen Count	Brand spoofing uses hyphens e.g. secure-paypal.com
4	HTTPS Status (0 or 1)	Legitimate sites use HTTPS; phishing often does not
5	IP Address Presence (0 or 1)	Phishing often navigates directly to an IP address
6	@ Symbol Presence (0 or 1)	@ in URL ignores everything before it
7	Domain Length	Phishing domains tend to be unusually long

8	Social Engineering Keyword Count	login, verify, update, secure, bank, account, support

The probabilities for each class are generated using the predict_proba method in the model and then the confidence percentage is calculated by multiplying max(probabilities) by 100. In case the ML model outputs a result indicating phishing, the ml_score becomes 60, which is added to the hybrid score. The coefficient of the machine learning model in the formula is 0.6, which is more than the coefficient

4.3.4 WHOIS Domain Age Checker

The check_domain_age() method uses the python-whois module to fetch the domain information. [20] Some special cases where care should be taken include:

- When the creation time is a list, we take the first element which is not null.
- If the creation time has time zone awareness, then it is first converted to UTC time to prevent errors due to the conversion of timezone aware datetimes since the timezone offset gets stripped off.
- The age of the domain in days is calculated as (datetime.utcnow() – creation).days.

Table 7: WHOIS Domain Age Thresholds

Age Range	Risk Level	WHOIS Score Added
Under 30 days	Very New Domain — HIGH RISK	+30
30 to 180 days	Recently Registered — SUSPICIOUS	+15
Over 180 days	Established Domain — OK	+0

4.3.5 Redirect Chain Tracer

The trace_redirects() function creates a session object with a maximum of 10 redirects and performs a GET call with allow_redirects set to True. Each intermediate step found in response.history is further classified using the _classify_hop() helper function as:

• SUSPICIOUS – Contains URL shortening sites (bit.ly, tinyurl.com, t.co, goo.gl, ow.ly, rebrand.ly)

• WARNING – HTTP (unencrypted) redirect

• ALERT – Redirect cycle detected (raises TooManyRedirects exception)

• UNREACHABLE – Failure to connect

• TIMEOUT – No response from server after 7 seconds

• OK – No problems detected

The redirect score is determined based on the sum of individual hop values (ALERT: 25, SUSPICIOUS: 15, WARNING: 10), 10 extra points if the hop chain contains more than three hops, and no more than 30 points total.

4.3.6 Campaign Detection Module

The workflow involved in detecting a campaign follows a two-step approach. First, the function store_url() is used to log the URL into the database under the 'UNKNOWN' verdict. Then, detect_campaign() runs its query involving COUNT(*), and since the URL was already logged in at this point, the ongoing scan will be counted in as well. Thus, if the value obtained from the query is three or higher, a campaign warning is generated. Once the entire hybrid analysis is done, the function _update_verdict() in analyzer.py is used to update the database record with the new verdict.[10]

4.3.7 QR Code Analysis Pipeline

The overall phishing detection process starts with calling the scan_qr_for_phishing() method that resides in the qr_scanner.py file. After scanning the QR code image by using an OCR tool known as QRCodeDetector from OpenCV in three steps (normal, gray, Otsu threshold), a URL inside the QR image is detected using a regular expression expression https?://[^\s]+. Once the URL is detected, it is checked if the URL belongs to any known secure QR scanner site such as scan.page, qr.io, flowcode.com, among others. In that case, the output is marked as SAFE. Otherwise, the URL goes into the full analysis via analyze_url().[8]

4.3.8 Hybrid Scoring Formula

The last hybrid score represents the core contribution made by PhishGuardX. All the scores have been combined into one through the equation below:

Final Score = round((rule_score x 0.4) + (redirect_score x 0.4) + (content_score x 0.4) + (whois_score x 0.4) + (ml_score x 0.6))

The final result will not exceed 100. The machine learning (ML) part has been given a higher coefficient (0.6) because of its training using real data. The rule, redirect, content, and WHOIS parts have received equal coefficients (0.4).

Table 8: Scoring Thresholds and Verdicts

Score Range	Verdict	Meaning
0 — 29	SAFE	No significant phishing indicators detected
30 — 69	SUSPICIOUS	Some indicators found, proceed with caution
70 — 100	PHISHING	High confidence phishing threat detected

4.3.9 Trusted Domain Fast-Path

Before processing the entire pipeline, the system first checks whether the provided URL exists in the set of fifteen trusted URLs, which include google.com, github.com, wikipedia.org, youtube.com, microsoft.com, apple.com, amazon.com, twitter.com, x.com, linkedin.com, reddit.com, instagram.com, facebook.com, and whatsapp.com. When there is a match, the system immediately outputs SAFE with a value of zero and does not proceed further with the other checks**.**

4.3.10 PDF Report Generation

The generate() method in the class ReportGenerator from reporter.py creates the PDF using this procedure:

• The Case ID (PHISH-YYYYMMDD-HHMMSS) along with the time stamp associated with it is produced.

• The analysis data is converted to JSON and a SHA-256 digest of it is calculated to ensure integrity of evidence.

• The cover page for the report with dark theme is designed by using ReportLab canvas instructions directly.

• The body pages are constructed using SimpleDocTemplate with the header/footer decorator being applied to all pages.

• The thirteen sections of the report are created by creating flowables (Paragraph, Table, Spacer, HRFlowable, PageBreak).

• The story to be printed is provided to doc.build().

• The body pages PDF and the cover page PDF are merged together using PyPDF and saved in reports/folders

Table 9: Report Section

#	Report Section
1	Executive Summary
2	Target Information
3	WHOIS Domain Intelligence
4	Machine Learning Analysis
5	Rule-Based Detection Findings
6	Redirect Chain Analysis
7	Content Analysis
8	QR Code Analysis
9	Campaign Intelligence
10	Attack Simulation and Kill Chain
11	Recommendations
12	Execution Logs
13	Cryptographic Evidence Integrity (SHA-256)

4.4 User Interface Screenshots

4.4.1 Tkinter Desktop GUI Application

The GUI application using Tkinter library is called using python gui.py. The following features are incorporated into the GUI application: header bar; drop-down list of input types (URL, email address, SMS, path to file, QR code); target text box; list of action buttons (Run Analysis, Show Report, Clear); section where verdict is shown (using color coding), risk score, machine learning confidence, and type of attack carried out; execution flow chart; and finally analysis logs.

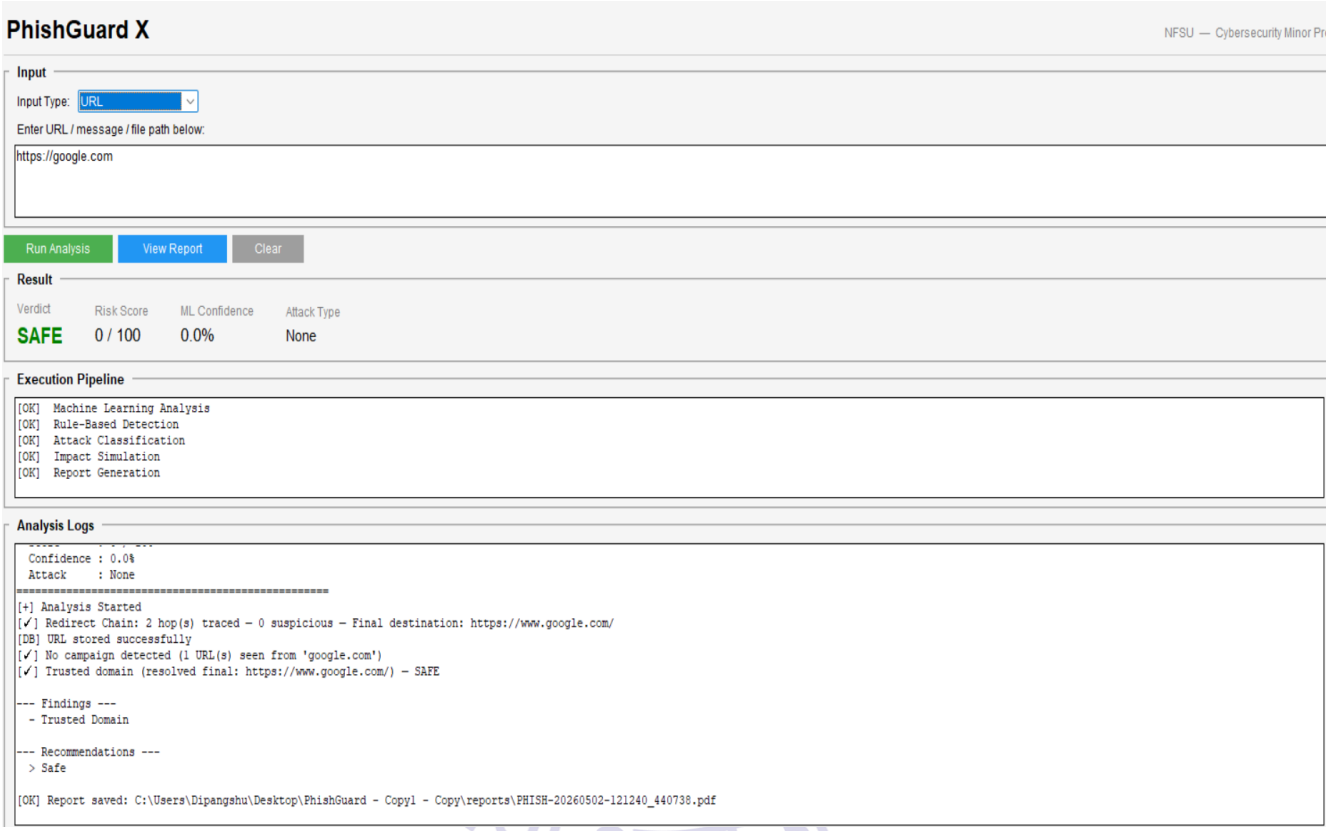

Figure 8: Tkinter Desktop GUI — Main Interface

4.4.2 Streamlit SOC Dashboard - Threat Scanner

The Streamlit application is started using the streamlit run dashboard.py command. The Threat Scanner interface includes an animated pipeline, risk score shown using the Plotly gauge widget, the conclusion badge, confidence level from machine learning, domain age, WHOIS data, findings labels, advice to follow, attack simulation procedure, and redirect chain details.

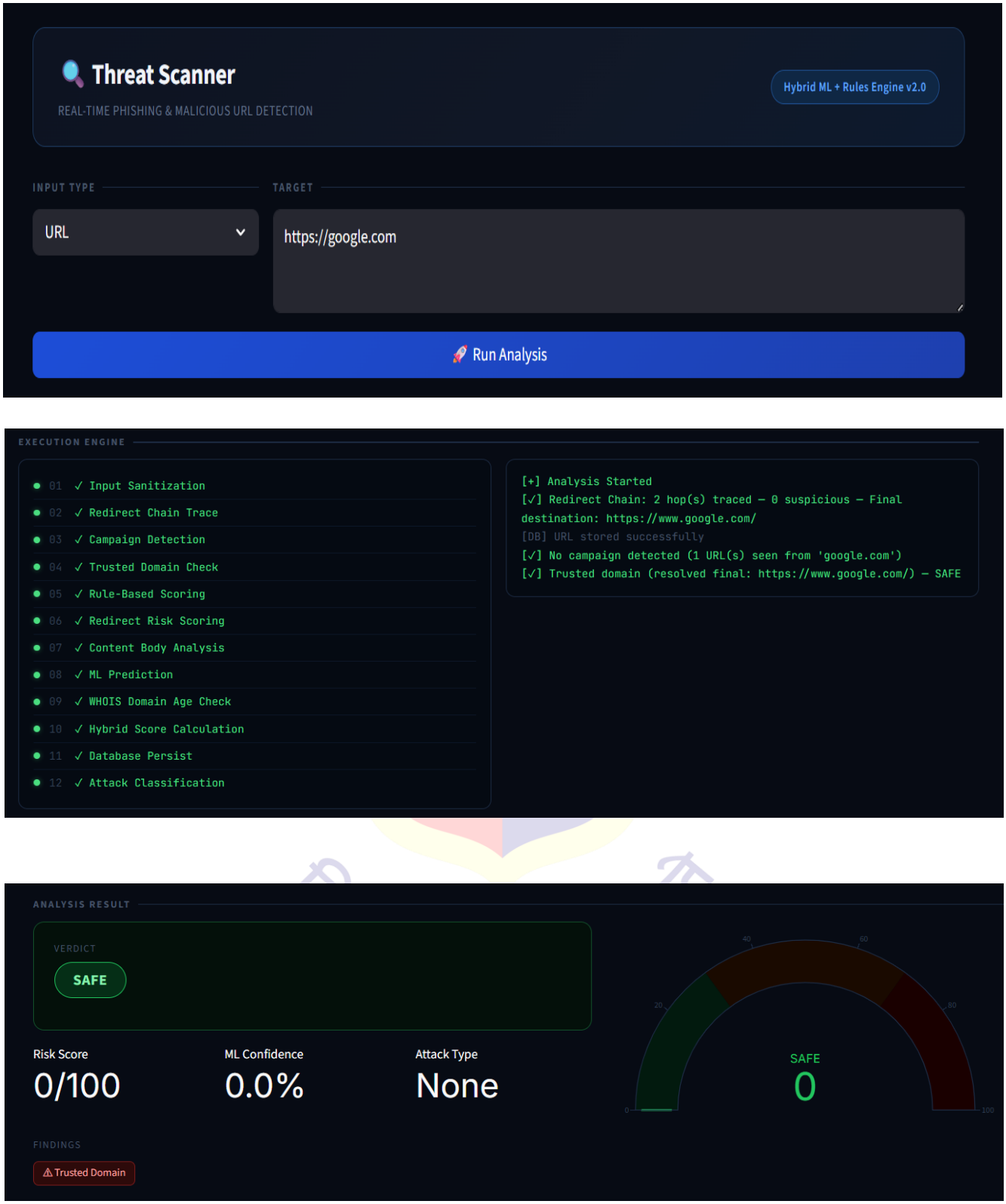

Figure 9: Streamlit SOC Dashboard - Threat Scanner Page

4.4.3 Analytics Page

There are five statistics cards on the Analytics page: Total Scans, Phishing, Suspicious, Safe, and Threat Rate. Three Plotly visualizations are also provided here: a donut pie chart showing the distribution of verdicts, a stacked bar graph showing daily scans, and a bar graph with the most scanned domains.

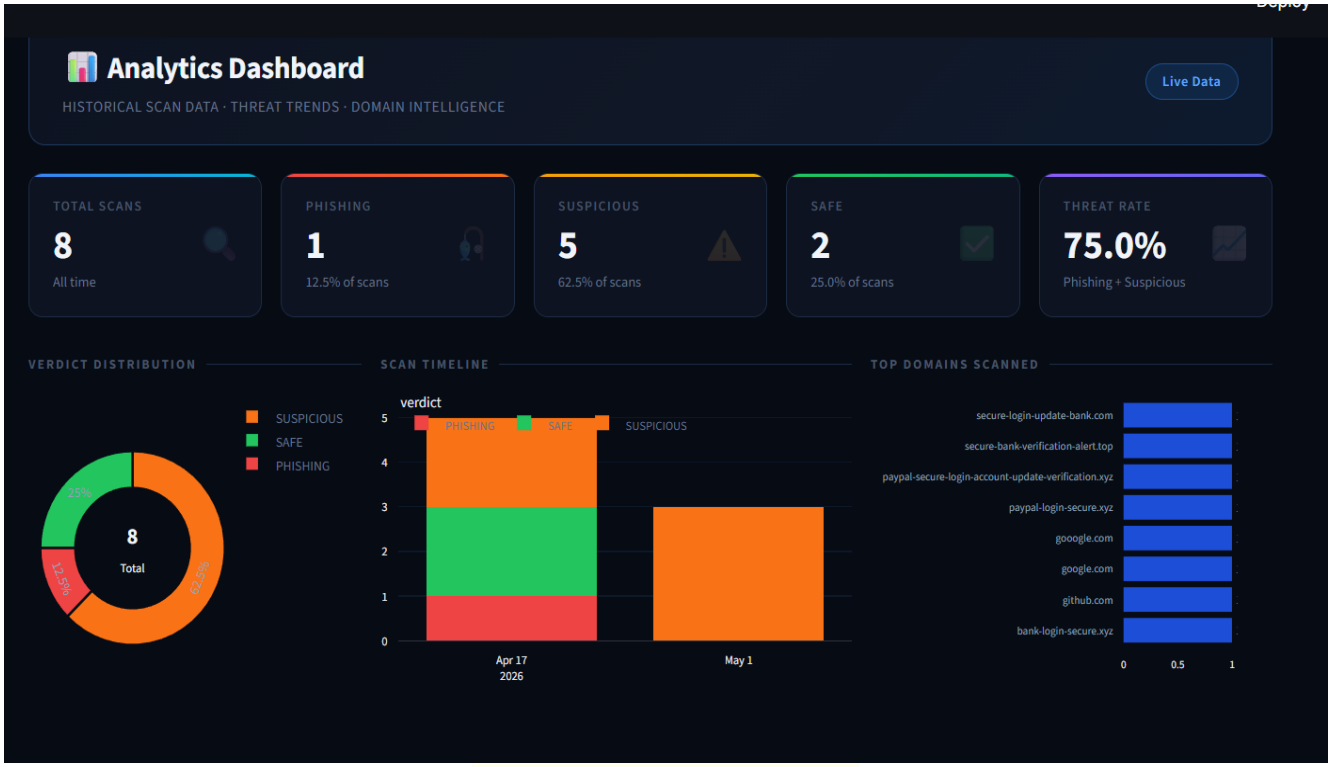

Figure 10: Streamlit Dashboard - Analytics Page with Charts

4.4.4 Campaign Monitor Page

The Campaign Monitor web page shows a list of all the domains which were spotted on at least three different URLs. The domain appears in its own collapsible section showing various statistics (Total, Phishing, Suspicious, Safe) along with a detailed table of all the URLs seen for that domain.

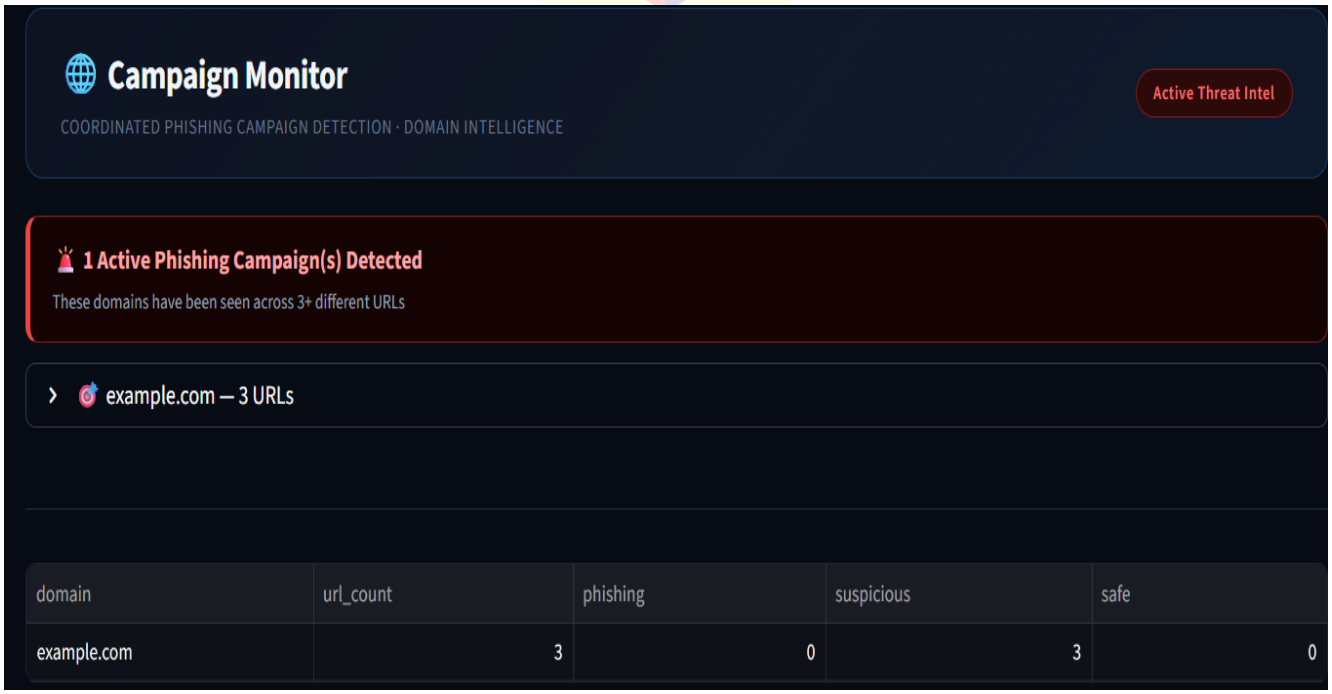

Figure 11: Campaign Monitor Page

4.4.5 Pdf Aalysis Report

The system generates a PDF report after each analysis. It includes the input details, final verdict, risk score, ML confidence, and attack type. The report also shows key findings, redirect information, and basic recommendations. This helps the user easily understand the result and keep a record of the analysis.

1. EXECUTIVE SUMMARY
High-level threat overview

VERDICT: SAFE

0 / 100

LOW RISK HIGH RISK

Report ID	PHISH-20260502-130658_491775
Timestamp	02 May 2026 13:06:58 UTC
Input Target	https://google.com
Verdict	SAFE
Risk Score	0 / 100
ML Confidence	0.0%
Attack Type	None
Campaign Detected	No

Figure 12: Sample PDF Analysis Report

CHAPTER 5 RESULTS AND FUTURE SCOPE

5.1 Advantages and Unique Features

5.1.1 True Hybrid Detection

The PhishGuardX tool uses six individual signals to build up a mathematical formula for the hybrid score rather than relying on one particular approach. In case one of these layers is bypassed, for example, by using an old enough domain that can avoid the WHOIS check, other layers like brand impersonation and machine learning scores will still factor into the final total.[1][3]

5.1.2 QR Code Phishing Detection

PhishGuardX is one of the handfuls of open-source solutions that can decode a QR code image and analyze the underlying URL for any potential phishing elements. PhishGuardX uses an iterative method for decoding QR code images, starting from standard decoding and then going to grayscale decoding, followed by Otsu thresholding, and finally, WeChatQRCode decoding.[8]

5.1.3 Coordinated Campaign Detection

The tracking system, which uses a SQLite database, captures all domains that have been scanned, raising an alert for any campaign if a particular domain has been linked to three or more different URLs. This feature is not commonly found in traditional URL checkers. It helps in identifying phishing campaigns where a hacker uses several URLs from one domain.[10]

5.1.4 Multi-Input Support

This allows for five modes of entry through a unified interface including: URLs, emails, SMS messages, text file entry, and QR codes. In instances where there are no URLs included in the email or SMS entry, a temporary tag such as ‘email-input’ and ‘sms-input’ is provided. This ensures that the URL history remains intact.

5.1.5 Professional Analysis Reports

The analysis report consists of thirteen chapters and makes use of the MITRE ATT&CK methodology along with the use of SHA-256 hashes, which is a rare feature that has not been implemented in other academic tools. The SHA-256 hash for the analysis done can be found in this very report, thus indicating that there have been no modifications to this report after its creation.[7][9]

5.1.6 Dual Interface Design

This application enables dual interfaces, which include Tkinter desktop graphical user interface (GUI), and Streamlit web-based GUI, hence allowing it to function in different settings. For example, instant individual checks can be made using Tkinter GUI, whereas continuous SOC monitoring, together with campaign visibility and historical analysis, can be carried out using the Streamlit web interface.[6]

5.2 Results and Discussion

The following types of inputs were considered in order to evaluate PhishGuardX: known phishing URLs, legitimate URLs from reliable domains, phishing e-mails that were crafted by the tester, SMS messages with urgency keywords, QR codes pointing to URLs, and various URLs belonging to one domain.

For testing the PhishGuardX, we used various types of input data such as known phishing websites, trusted sites on verified domains, phishing emails, urgent text SMS with links, malicious sites in QR codes, and a chain of URLs on one site. In URL classification testing, the system correctly determined all trusted domains (such as google.com and github.com) as SAFE by using the trusted-domain fast path technique and gave a score of 0. For new domains which mimic well-known brand names, the total scoring (brand mimicry +30, suspicious top-level domain +20, WHOIS age +30) plus ML prediction gave scores more than 70 and consequently a PHISHING decision.[1][2]

Content-based analysis was performed on the e-mail, which accurately captured elements such as indications of urgency ("within 24 hours," "act now," "final notice"), brand spoofing phrases ("dear customer," "Microsoft," "PayPal"), and calls-to-action CTA expressions ("click here," "verify now," "reset your password"). The content score on its own could go up to 30 points, for example.

In terms of QR decoding, the OpenCV decoder worked successfully on good and corrupted QR codes in multiple test cases. Well-known QR decoding sites such as scan.page and flowcode.com were properly whitelisted and gave SAFE verdicts straight away.[8]

As far as processing of QR codes is concerned, the decoder built on OpenCV has demonstrated efficiency for high-quality QR images as well as those that could be considered less quality. White-listed QR applications such as scan.page and flowcode.com have shown correct processing outcomes. The algorithm for detecting campaigns has demonstrated that it has correctly identified the test domain based on the scanning of the third URL coming from the same domain. The sequence of store_url() and detect_campaign() algorithms ensured the fact that by the third scan, the information was saved to the database.

All input cases were processed successfully to generate the PDF analysis reports. Hash values were unique for each analysis, which confirmed that the documents were unique. The MITRE ATT&CK

framework correctly mapped the attacks on credential harvesting (T1566, T1056, T1539) and financial scam attacks (T1566, T1657).[9]

5.3 Future Scope

5.3.1 VirusTotal And Shodan API Integration

Integration of PhishGuardX with third-party threat intelligence APIs, such as the multi-engine URL reputation check from VirusTotal and IP intelligence from Shodan, will significantly improve detection accuracy, especially for IP-based phishing sites. This can be done easily due to the current system design that supports this feature.

5.3.2 Browser Extension

An extension on the browser that calls the PhishGuardX backend API and shows a badge for each link upon hovering would make this solution more accessible to non-technical users. The Streamlit Dashboard backend can be provided as a REST API endpoint.

5.3.3 Real-time Email Inbox Scanning

Using Google mail or Microsoft outlook APIs, PhishGuardX can automatically scan every new email received, which would reduce the manual step of pasting the email message into the application by the user.

5.3.4 Deep Learning Model

The current Random Forest algorithm may be replaced or supplemented by a carefully tuned BERT or CharCNN algorithm that processes the URL directly in its character format. This method may have the capability of improving adversarial URL detection because such URLs are often designed to fool feature-based models.[16]

5.3.5 Screenshot Capture and Visual Analysis

Functionality that enables the capturing of screenshots for the web pages and comparing their visual characteristics with those of known brand homepages using methods such as perceptual hashing and/or CNN can be used for identifying any instances of brand impersonation even if the URL looks authentic.

5.3.6 Alert Notifications

In SOC implementation, the integration of notifications through channels such as Slack, Discord, or Microsoft Teams webhooks helps send immediate alerts when a decision of PHISHING is made or when a campaign is detected, thus eliminating the requirement to monitor the dashboard constantly.

CHAPTER 6 CONCLUSION

This work has managed to develop a multilayer hybrid system for phishing detection known as PhishGuardX. This system effectively tackles the core weaknesses of current phishing detection systems through a unique integration of six different detection techniques into a single hybrid score generator, providing an insightful risk score and professional analysis for each analysis.

The rule-based engine detects the structure-based phishing cues, which consistently appear in phishing campaigns and include brand spoofing, suspicious TLDs, URL shorteners, IPs, hexadecimal coding, and lack of HTTPS. The machine learning classifier generates probabilistic output based on empirical phishing data, allowing generalization beyond inflexible rule sets. [1][3][5]The WHOIS domain age check engine detects a common trend in modern phishing campaigns - the registration of newly created domains, which have never before been included in any blacklists. The redirect chain tracker finds the techniques used by phishers, whereby they create a chain of redirections using URL shorteners and HTTP redirection to conceal the final malicious link. The content analyzer evaluates the psychological manipulation language used in emails and SMS messages, which aims to force victims into action.[20][4]

Scanning QR codes is intended to deal with the increasing problem of quishing, a technique which most of today's tools fail to address. The double interface, involving the use of Tkinter for the desktop-based GUI interface for quick scans, and Streamlit for the SOC-based dashboard, ensures that the tool is versatile and can be used by individuals with different technical skills.[8][6]

The PDF report generator produces a professional report comprising of 13 sections including MITRE ATT&CK techniques mapping as well as the use of SHA-256 hash function to ensure the integrity of evidence. The hash function ensures that all reports produced using PhishGuardX can always be checked for any alterations.[7][9]

To conclude, it is evident from PhishGuardX that a properly designed Python program with appropriate modules can offer phishing identification and notification without having to rely on expensive software or even costly APIs or services available on the cloud. Moreover, the project presents a solid starting point towards further improvements, such as the inclusion of threat intelligence feeds, real-time email scanning, and machine learning-based identification. This project also shows how cybersecurity tools can be created and understood by students.

REFERENCES

[1] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Systems with Applications, vol. 117, pp. 345–357, 2019. https://doi.org/10.1016/j.eswa.2018.09.029

[2] R. M. Mohammad, F. Thabtah, and L. McCluskey, "Predicting phishing websites based on self-structuring neural network," Neural Computing and Applications, vol. 25, no. 2, pp. 443–458, 2014.
https://doi.org/10.1007/s00521-013-1490-z

[3] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001. https://doi.org/10.1023/A:1010933404324

[4] Anti-Phishing Working Group (APWG), Phishing Activity Trends Report Q1–Q4 2023, 2024. [Online]. Available:
https://apwg.org/trendsreports/

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[6] Streamlit Inc., Streamlit Documentation — Build Data Apps in Python, 2023. [Online]. Available: https://docs.streamlit.io/

[7] ReportLab Ltd., ReportLab User Guide — PDF Generation Library for Python, 2023. [Online]. Available:
https://www.reportlab.com/docs/reportlab-userguide.pdf

[8] OpenCV Team, OpenCV QR Code Detection and Decoding, 2023. [Online]. Available: https://docs.opencv.org/4.x/de/dc3/classcv_1_1QRCodeDetector.html

[9] MITRE Corporation, MITRE ATT&CK Framework — Phishing Techniques (T1566), 2023. [Online]. Available: https://attack.mitre.org/techniques/T1566/

[10] Python Software Foundation, sqlite3 — DB-API 2.0 Interface for SQLite Databases, 2023. [Online]. Available: https://docs.python.org/3/library/sqlite3.html

[11] B. B. Gupta, A. Tewari, A. K. Jain, and D. P. Agrawal, "Fighting against phishing attacks: State of the art and future challenges," Neural Computing and Applications, vol. 28, no. 12, pp. 3629–3654, 2017. https://doi.org/10.1007/s00521-016-2275-y

[12] M. Khonji, Y. Iraqi, and A. Jones, "Phishing detection: A literature survey," IEEE Communications Surveys & Tutorials, vol. 15, no. 4, pp. 2091–2121, 2013. https://doi.org/10.1109/SURV.2013.032213.00009

[13] Verizon, 2024 Data Breach Investigations Report, 2024. [Online]. Available: https://www.verizon.com/business/resources/reports/dbir/

[14] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," arXiv preprint arXiv:1802.03162, 2018. https://arxiv.org/abs/1802.03162

[15] P. Prakash, M. Kumar, R. R. Kompella, and M. Gupta, "PhishNet: Predictive blacklisting to detect phishing attacks," in Proc. IEEE INFOCOM, pp. 346–350, 2010. https://doi.org/10.1109/INFCOM.2010.5462216

[16] A. Blum, B. Wardman, T. Solorio, and G. Warner, "Lexical feature based phishing URL detection using online learning," in Proc. 3rd ACM Workshop on Artificial Intelligence and Security (AISec), pp. 54–60, 2010.
https://dl.acm.org/doi/10.1145/1866423.1866434

[17] A. A. Ubing, S. K. B. A. Jasmi, A. Abdullah, N. Z. Jhanjhi, and M. Supramaniam, "Phishing website detection: An improved accuracy through feature selection and ensemble method," International Journal of Advanced Computer Science and Applications, vol. 10, no. 1, 2019. https://doi.org/10.14569/IJACSA.2019.0100133

[18] S. C. Jeeva and E. B. Rajsingh, "Intelligent phishing URL detection using association rule mining," Human-centric Computing and Information Sciences, vol. 6, no. 1, pp. 1–19, 2016. https://doi.org/10.1186/s13673-016-0064-3

[19] Python Software Foundation, urllib.parse — Parse URLs into Components, 2024. [Online]. Available:
https://docs.python.org/3/library/urllib.parse.html

[20] R. Johari, A. Kumar, and P. Gupta, "WHOIS-based phishing site detection using machine learning," Procedia Computer Science, vol. 167, pp. 781–792, 2021. https://doi.org/10.1016/j.procs.2020.03.392

Bibilography

The following references were referred to in the process of conducting research and designing the concept for the proposed PhishGuardX system**.**

[1] Gupta, B. B., Tewari, A., Jain, A. K., & Agrawal, D. P. (2017). Fighting against phishing attacks: State of the art and future challenges. Neural Computing and Applications, 28(12), 3629–3654. https://doi.org/10.1007/s00521-016-2275-y

[2] Khonji, M., Iraqi, Y., & Jones, A. (2013). Phishing detection: A literature survey. IEEE Communications Surveys & Tutorials, 15(4), 2091–2121.

https://doi.org/10.1109/SURV.2013.032213.00009

[3] Verizon. (2024). 2024 Data Breach Investigations Report. Retrieved from https://www.verizon.com/business/resources/reports/dbir/

[4] Le, H., Pham, Q., Sahoo, D., & Hoi, S. C. H. (2018). URLNet: Learning a URL representation with deep learning for malicious URL detection. arXiv preprint arXiv:1802.03162. https://arxiv.org/abs/1802.03162

[5] Prakash, P., Kumar, M., Kompella, R. R., & Gupta, M. (2010). PhishNet: Predictive blacklisting to detect phishing attacks. In Proceedings of IEEE INFOCOM, 346–350. https://doi.org/10.1109/INFCOM.2010.5462216

[6] Blum, A., Wardman, B., Solorio, T., & Warner, G. (2010). Lexical feature based phishing URL detection using online learning. In Proceedings of the 3rd ACM Workshop on Artificial Intelligence and Security (AISec), 54–60.

[7] Ubing, A. A., Jasmi, S. K. B. A., Abdullah, A., Jhanjhi, N. Z., & Supramaniam, M. (2019). https://dl.acm.org/doi/10.1145/1866423.1866434

[8] Phishing website detection: An improved accuracy through feature selection and ensemble method. International Journal of Advanced Computer Science and Applications, 10(1). https://doi.org/10.14569/IJACSA.2019.0100133

[9] Jeeva, S. C., & Rajsingh, E. B. (2016). Intelligent phishing URL detection using association rule mining. Human-centric Computing and Information Sciences, 6(1), 1–19. https://doi.org/10.1186/s13673-016-0064-3

[10] Python Software Foundation. (2024). urllib.parse — Parse URLs into components. Retrieved from https://docs.python.org/3/library/urllib.parse.html

[11] Johari, R., Kumar, A., & Gupta, P. (2021). WHOIS-based phishing site detection using machine learning. Procedia Computer Science, 167, 781–792. https://doi.org/10.1016/j.procs.2020.03.392

[12] Johari, R., Kumar, A., & Gupta, P. (2021). WHOIS-based phishing site detection using machine learning. Procedia Computer Science, 167, 781–792. https://doi.org/10.1016/j.procs.2020.03.392

PROGRESS REPORT

The report below captures the progress made during the implementation of the minor project entitled "PhishGuardX: Hybrid Phishing Detection System," which was conducted between January 2026 and May 2026 through supervision by the project supervisor at National Forensic Sciences University, Goa Campus.

January 2026

The topic for the project was chosen, and the problem statement was formulated. Literature research and analysis were conducted in regard to existing solutions to detect phishing emails, relevant scientific publications, and open-source initiatives on this matter. Project outline, which included the scope of the project and features list, was prepared.

February 2026

The system architecture was designed and the modular design of the project was planned out. The environment for development was created which involved setting up Python libraries like Scapy, scikit-learn, Streamlit, and ReportLab. Key modules such as the feature extractor, rule-based analyzer, and the Random Forest classifier training process were implemented individually and were tested in a stand-alone manner. The dataset containing URLs labeled accordingly was created for the machine learning algorithm to learn from initially.

March 2026

The WHOIS domain age checker, redirect chain tracer, content analysis engine, and campaign detection module were implemented and integrated into the central hybrid scoring engine. The QR code scanning capability using OpenCV was added and tested using sample QR code images. The hybrid scoring formula combining all six detection signals was finalized and validated against a set of known phishing and legitimate test URLs

April 2026

Tkinter GUI and Streamlit SOC Dashboard were developed and integrated with the common backend engine. The report generation module utilizing ReportLab was developed for all 13 report sections, incorporating MITRE ATT&CK framework mapping and evidence hashing using SHA-256. The system was tested end-to-end with different input types, such as URL, email, SMS, and QR codes. Bug fixes and handling of edge cases, such as failed WHOIS queries and redirect loops, were implemented.

May 2026

Final project report has been prepared, proofread, and formatted according to university guidelines.All relevant graphs, diagrams, snapshots, and charts have been included within the report. Sections such as references, bibliography, and appendices have also been added. Final revisions have been made after review from the supervisor. Completed project report, together with all source codes, has been submitted as part of Minor Project Assessment of M.Sc. Cyber Security.

PLAGIARISM REPORT